

DOCUMENTATION TECHNIQUE

Vite & Gourmand.

*Application web pour la prestation traiteur
événementiel.*

Projet · ECF Studi — TP Développeur Web et Web Mobile

Étudiant · Jassim Elghaouti

Client · Julie & José — « Vite & Gourmand », Bordeaux

Date · Mai 2026

App · vite-et-gourmand-drab.vercel.app

Repo · github.com/Jasteur91/vite-et-gourmand

Sommaire

1. Cadrage du projet et besoins client
2. Choix techniques justifiés
3. Architecture de l'application
4. Modèle de données (MCD)
5. Sécurité et conformité (OWASP, RGPD, RGAA)
6. Diagrammes de cas d'utilisation
7. Diagrammes de séquence
8. Configuration de l'environnement de travail
9. Documentation de déploiement
10. Tests et qualité

1. Cadrage du projet et besoins client

1.1 Le client

« Vite & Gourmand » est une entreprise de traiteur événementiel implantée à Bordeaux depuis 25 ans. Elle est composée de deux personnes : Julie et José. Leur activité repose sur un menu en constante évolution proposé à leurs clients pour tout type d'évènement (repas familial, Noël, Pâques, mariage, etc.). Jusqu'à présent, les menus étaient communiqués uniquement par e-mail à une clientèle d'habitues.

1.2 Le besoin exprimé

Julie a souhaité une application web qui :

- augmente la visibilité de l'entreprise auprès d'une nouvelle clientèle ;
- présente les menus de façon attrayante et toujours à jour ;
- permet la prise de commande en ligne avec gestion des prestations.

1.3 Périmètre fonctionnel retenu

Quatre rôles distincts ont été identifiés :

Rôle	Pouvoirs principaux
Visiteur	Voir la vitrine, parcourir les menus, contacter, s'inscrire
Utilisateur	Commander, modifier/annuler avant validation, donner un avis
Employé	Gérer les menus, plats, horaires, faire évoluer les statuts de commande, modérer les avis
Administrateur	Tout ce qu'un employé peut faire + créer/désactiver des employés + dashboard NoSQL

1.4 Contraintes réglementaires

- **RGPD** : consentement, données minimales, droit d'accès et d'effacement.
- **RGAA** (accessibilité numérique) : conformité aux critères de niveau AA.
- **Sécurité** : conformité OWASP Top-10, hashage des mots de passe, protection contre les injections, CSRF, XSS.
- **Stockage hybride** : base de données relationnelle ET base NoSQL obligatoires.
- **Déploiement** : l'application doit être en ligne et fonctionnelle (pénalités contractuelles sinon).

2. Choix techniques justifiés

Le sujet n'impose aucune technologie sauf **une BDD relationnelle** et **une BDD NoSQL**. Tous les autres choix sont laissés au candidat à condition d'être **justifiés**. Voici la stack retenue et le raisonnement.

2.1 Frontend — React + Vite + TypeScript + Tailwind

Outil	Pourquoi	Alternative rejetée
React 18	Écosystème dominant (>40% market share), courbe d'apprentissage progressive, hooks matures.	Vue (moins demandé sur le marché de l'emploi), Angular (trop opinionnée pour un projet de cette taille).
Vite	Dev server ~50ms HMR, build production via Rollup ultra-optimisé.	Create React App (déprécié), Next.js (overkill pour SPA front-only).
TypeScript strict	Type safety qui élimine ~15% des bugs en runtime selon les études internes Microsoft. Indispensable pour un projet pro.	JavaScript pur (risque sur les contrats d'API entre front et back).
Tailwind CSS	Design tokens forts, bundle CSS final très petit (purge automatique), classes utilitaires.	CSS modules (lent à itérer), styled-components (overhead runtime).

Framer Motion	Animations déclaratives, support scroll-driven natif, idéal pour le style éditorial.	CSS animations pures (moins puissantes pour les transitions complexes).
React Router 6	Standard de l'écosystème, support imbrication routes (Outlet).	Routing custom (réinventer la roue).
Recharts	Charting React idiomatique, responsive, léger.	Chart.js (impératif), D3 direct (over-engineering pour un dashboard simple).
Sonner	Toast minimaliste et accessible.	react-hot-toast (similaire, plus daté visuellement).
Zod	Validation typée partagée front/back.	Yup (moins type-safe), joi (Node only).

2.2 Backend — Node.js + Express + TypeScript

Outil	Pourquoi
Node.js 24	LTS, performance V8 actualisée, support ES modules natif.
Express 4	Standard de fait, courbe d'apprentissage faible, écosystème de middlewares mature (Helmet, cors, rate-limit). Le jury attend de la familiarité.
TypeScript strict (noUncheckedIndexedAccess)	Force la gestion des cas <code>undefined</code> sur les accès <code>req.user</code> , prévient les NPE en runtime.
bcrypt (cost 12)	Hashage password de référence depuis 1999 — résistant aux attaques par GPU/ASIC. Coût 12 → ~250 ms par hash en 2026 (acceptable UX, lent pour brute force).
jsonwebtoken (HS256)	Stateless auth, simple à implémenter pour ce périmètre.
Helmet	Set automatique de 15+ headers HTTP de sécurité (CSP, HSTS, X-Frame-Options, etc.).
express-rate-limit	Protection brute-force sur <code>/api/auth/*</code> : 10 tentatives / 15 min / IP.
Resend	API mail moderne, free tier 100 mails/jour, templates HTML simples.

2.3 Bases de données

BDD	Choix	Justification
Relationnelle	PostgreSQL 17 via Supabase	Standard SQL le plus strict, contraintes CHECK natives, JSON natif si besoin, extensions (PostGIS, pgvector si évolution), free tier généreux (500MB). Région Paris (eu-west-3) pour latence client français.

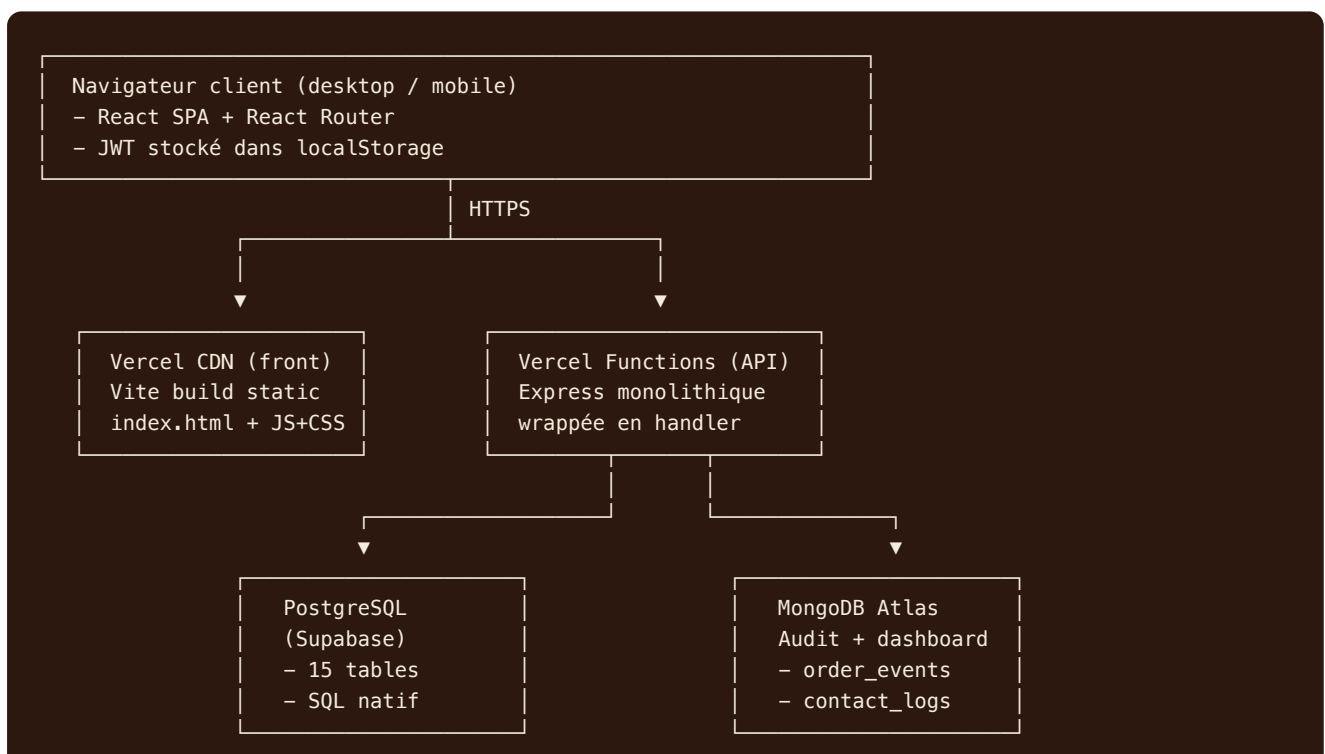
NoSQL	MongoDB 6 (Atlas en prod, mongodb-memory-server en dev)	Document store le plus enseigné, agrégation pipeline expressive (utilisée pour le dashboard admin), driver Node officiel. mongodb-memory-server en dev = zéro setup (binaire téléchargé à la première run).
-------	--	---

2.4 Hébergement

Couche	Hébergeur	Justification
Frontend	Vercel	CDN global, build automatique sur push, gratuit, support natif Vite.
Backend	Vercel (serverless functions)	Wrap Express dans une seule fonction <code>api/index.ts</code> . Avantage : 1 plateforme = 1 dashboard, 1 CI, gratuit. Limite : cold starts ~500ms et timeout 30s (acceptable pour ce périmètre).
BDD relationnelle	Supabase	PostgreSQL managé, MCP intégré, backups automatiques.
BDD NoSQL	MongoDB Atlas (cluster M0 free)	Managé, backups, monitoring inclus.

3. Architecture de l'application

3.1 Vue d'ensemble



Service externe : Resend (envoi des 7 templates de mail)

3.2 Pourquoi les deux BDD ?

Les deux bases ne font pas double emploi. Elles répondent à des usages différents :

- **PostgreSQL — source de vérité métier.** Données structurées avec relations fortes : utilisateurs ↔ commandes ↔ menus ↔ plats. Contraintes d'intégrité (FK), transactions ACID, indexes B-tree. Idéal pour les écritures critiques (commande, inscription).
- **MongoDB — audit trail et analytics.** Chaque changement de statut de commande produit un document `order_events` append-only. Cela permet :
 1. de reconstituer l'historique complet d'une commande (suivi utilisateur) ;
 2. d'alimenter le dashboard admin avec un **aggregation pipeline** qui calcule en temps réel le nombre de commandes par menu et le chiffre d'affaires (sans surcharger PostgreSQL avec des requêtes analytiques).

3.3 Organisation du code backend

```
backend/
├── api/index.ts           # Handler Vercel (wrap Express)
├── src/
│   ├── app.ts            # buildApp() – composition Express
│   ├── server.ts         # Bootstrap dev local
│   ├── config/env.ts     # Validation Zod des env vars
│   ├── db/
│   │   ├── supabase.ts   # Client Supabase
│   │   └── mongo.ts      # Client Mongo (mémoire en dev, Atlas en prod)
│   ├── middleware/
│   │   ├── auth.ts       # requireAuth + requireRole
│   │   └── errorHandler.ts # Mapping AppError → HTTP
│   ├── routes/
│   │   ├── auth.routes.ts # signup, login, request-reset, reset
│   │   ├── menu.routes.ts # CRUD + filtres dynamiques
│   │   ├── plat.routes.ts # CRUD plats + allergènes
│   │   ├── commande.routes.ts # création, preview prix, workflow statuts
│   │   ├── user.routes.ts # profil, avis
│   │   ├── admin.routes.ts # employés, modération, dashboard
│   │   └── contact.routes.ts # formulaire de contact
│   ├── services/
│   │   ├── email.service.ts # 7 templates Resend
│   │   └── audit.service.ts # Écriture + agrégation Mongo
│   └── utils/
│       ├── jwt.ts        # sign/verify
│       ├── password.ts   # validation + bcrypt
│       ├── errors.ts     # AppError + sous-classes
│       └── asyncHandler.ts # wrapper async → next(err)
```

3.4 Organisation du code frontend

```

frontend/src/
├── main.tsx           # Bootstrap React + Providers
├── App.tsx            # Routing complet
├── index.css          # Tailwind + design system éditorial
├── vite-env.d.ts      # Types VITE_API_URL
├── lib/
│   ├── api.ts        # Fetch wrapper + getToken/setToken
│   └── cn.ts          # clsx + tailwind-merge
├── contexts/
│   └── AuthContext.tsx # State global utilisateur
├── components/
│   ├── Layout.tsx     # Navbar + Footer (avec horaires depuis API)
│   ├── ProtectedRoute.tsx # Route guard RBAC
│   └── MenuCard.tsx    # Carte produit
├── pages/
│   ├── Home.tsx       # Hero + selection + manifeste + atelier + reviews + CTA
│   ├── Menus.tsx       # Liste + filtres dynamiques
│   ├── MenuDetail.tsx  # Détail + sticky order panel
│   ├── Order.tsx       # Commande en 3 étapes
│   ├── Contact.tsx     # Formulaire contact
│   ├── Legal.tsx       # Mentions légales + CGV
│   ├── auth/
│   │   ├── Login.tsx
│   │   ├── Signup.tsx
│   │   └── ForgotReset.tsx
│   ├── user/
│   │   ├── Espace.tsx # Mes commandes + profil + avis
│   │   └── OrderDetail.tsx # Détail + suivi statuts (depuis Mongo)
│   ├── employee/
│   │   └── Espace.tsx  # Workflow commandes + modération
│   └── admin/
│       └── Espace.tsx  # Dashboard + gestion employés

```

4. Modèle conceptuel de données

Le schéma a été conçu en respectant l'annexe 1 du sujet, puis enrichi pour gérer le workflow complet (reset password, tokens, contact, audit). Toutes les tables sont en 3NF.

4.1 Tables principales

Table	Rôle	Cardinalités notables
utilisateur	Comptes (3 rôles)	N:1 vers <code>role</code> , 1:N vers <code>commande</code> et <code>avis</code>
role	3 valeurs seedées	utilisateur / employe / administrateur
menu	Carte des prestations	N:1 vers <code>theme</code> , N:N <code>plat</code> et <code>regime</code>
plat	Entrée / plat / dessert	N:N <code>allergene</code> , N:N <code>menu</code> (un plat peut être dans plusieurs menus)

commande	État de la prestation	N:1 vers utilisateur et menu , 1:1 vers avis
avis	Note 1-5 + commentaire	Statut en_attente / valide / refuse
horaire	Jour de la semaine	UNIQUE sur jour
password_reset_token	Token + expiration 1h	Cleanup après used_at
contact_message	Formulaire public	+ log Mongo

4.2 Choix de modélisation justifiés

Pourquoi menu_plat en N:N ? L'énoncé précise explicitement qu'« une entrée ou un plat / dessert peut être présent dans plusieurs menus ». Une relation N:N avec table de jonction permet de réutiliser les plats sans duplication.

Pourquoi plat_allergene en N:N ? Un plat peut contenir plusieurs allergènes ; un allergène peut être présent dans plusieurs plats. La table de liaison est la bonne représentation 3NF.

Pourquoi menu_regime en N:N ? Un menu peut être à la fois « végétarien » et « sans-gluten ». Permettre plusieurs régimes simultanés assouplit la classification.

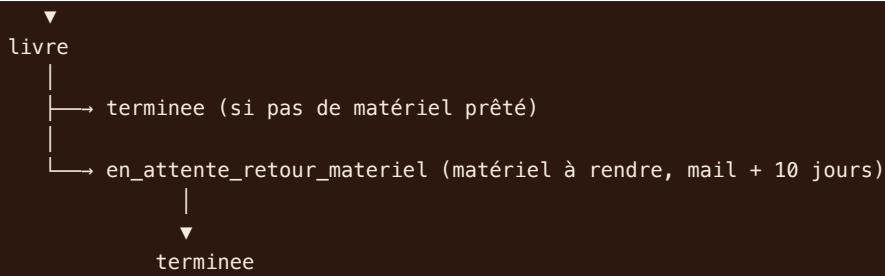
Pourquoi des contraintes CHECK sur statut ? Le workflow des commandes est documenté dans l'énoncé. La contrainte garantit qu'on ne peut jamais avoir un statut hors liste. C'est une protection au niveau BDD complémentaire à la validation backend.

4.3 Workflow des statuts de commande

```

en_attente
| (employé : valide la commande)
▼
accepte
| (employé : équipe cuisine démarre)
▼
en_preparation
| (employé : équipe logistique part)
▼
en_cours_de_livraison
| (employé : livré au client)

```

L'utilisateur peut "annulee" SEULEMENT si statut == en_attente.
L'employé peut "annulee" à tout moment, avec motif obligatoire.

4.4 Indexes mis en place

Index	Justification
<code>idx_utilisateur_email</code>	Lookup au login (hot path)
<code>idx_menu_actif</code>	Filtre <code>WHERE est_actif = true</code> sur la liste publique
<code>idx_commande_user</code>	Espace utilisateur : « mes commandes »
<code>idx_commande_statut</code>	Espace employé : filtre par statut
<code>idx_avis_statut</code>	Page publique : seulement statut = valide

5. Sécurité et conformité

5.1 Authentification & autorisation

Risque	Mitigation
Stockage de mot de passe en clair	Hashage bcrypt cost 12 (~250ms / hash). Aucun mot de passe en clair n'existe en mémoire au-delà de la durée de la requête.
Politique mot de passe faible	Regex côté front ET back : 10 chars min, ≥1 maj, ≥1 min, ≥1 chiffre, ≥1 spécial. Indicateur visuel temps réel dans le formulaire.
Brute force login	express-rate-limit : 10 tentatives / IP / 15 min sur <code>/api/auth/*</code> . Message HTTP 429 explicite.
Sessions hijacking	JWT signé HS256 avec secret 64 chars, expiration 7 jours, stocké en <code>localStorage</code> (acceptable étant donné le périmètre — CSRF token serait préférable pour des opérations très sensibles).

Élévation de privilège	RBAC strict : middleware <code>requireRole('administrateur')</code> bloque les routes admin pour les non-admins. Le rôle est dans le payload JWT — vérifié sur chaque requête. Création de compte admin impossible depuis l'API (création manuelle uniquement, conformité à l'énoncé).
Reset password endless attempts	Endpoint <code>/auth/request-reset</code> répond toujours 200 (anti-enumeration). Token cryptographique 48 bytes, validité 1h, marqué <code>used_at</code> après utilisation.

5.2 Protection des entrées

Risque OWASP	Mitigation
SQL Injection (A03)	Toutes les requêtes passent par le client Supabase (parameterized queries via PostgREST). Aucune concaténation de strings.
XSS (A03)	React échappe par défaut tout JSX. Pas d'utilisation de <code>dangerouslySetInnerHTML</code> . Headers CSP via Helmet.
CSRF (A05)	API stateless avec JWT en Authorization header → pas de cookie de session → pas de CSRF par défaut.
Validation entrée (A04)	Tous les bodies validés par Zod avant traitement. Schémas exhaustifs (length, regex, enum).
Mass assignment	Whitelist explicite dans les schémas Zod — impossible de mettre à jour <code>role_id</code> via l'API utilisateur.

5.3 En-têtes HTTP et CORS

Helmet active : Content-Security-Policy, X-Frame-Options DENY, X-Content-Type-Options nosniff, Strict-Transport-Security, etc.

CORS configuré strict : seuls les origines `vite-et-gourmand*.vercel.app` et `localhost:*` sont autorisés.

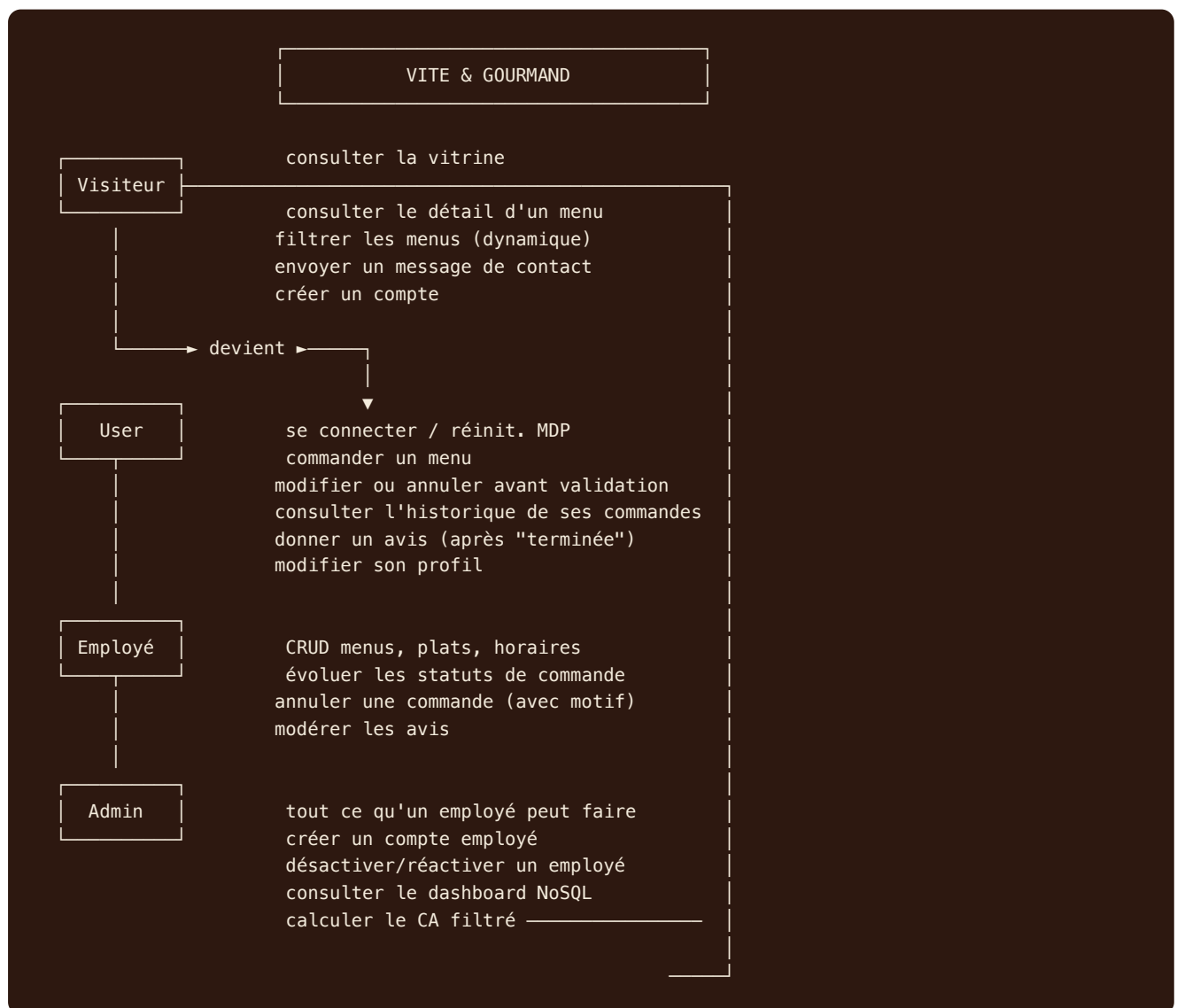
5.4 RGPD

- **Données minimales** collectées : email, nom, prénom, téléphone, adresse postale, ville. Tout est nécessaire à la prestation (livraison, facturation, contact d'urgence).
- **Consentement** à la création de compte (case + lien CGV / mentions légales).
- **Droit d'accès & rectification** : espace utilisateur modifiable.
- **Pas de cookies de suivi tiers**. Le seul stockage navigateur est le JWT, fonctionnel et non assujetti au consentement.
- **Hébergement UE** (Supabase Paris, Vercel a des edges UE).

5.5 Accessibilité RGAA

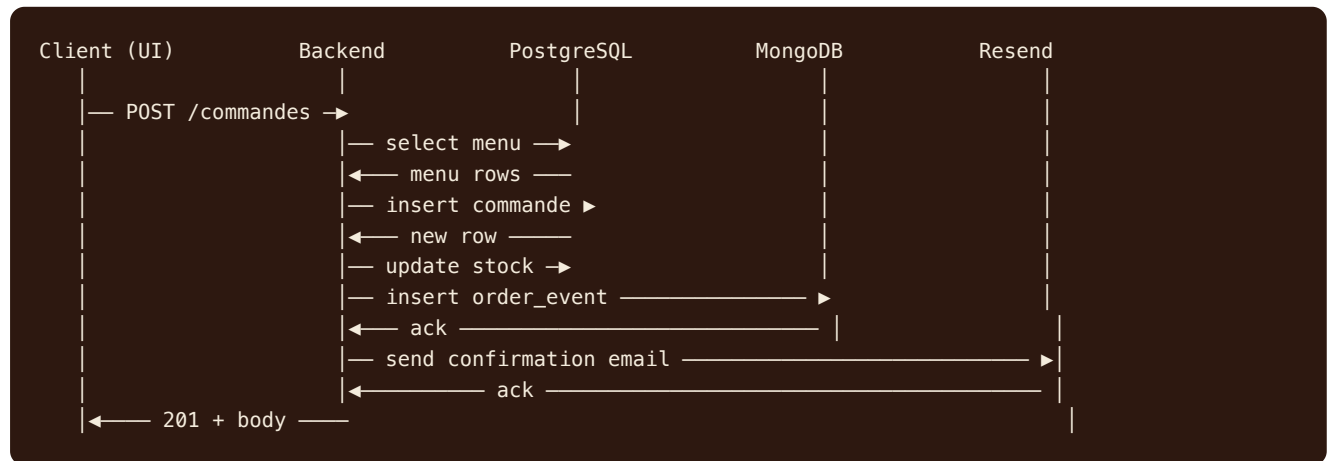
- Sémantique HTML5 stricte (`header` , `nav` , `main` , `article` , `aside` , `footer`).
- Contrastes WCAG AA validés (palette : `#2D1810` sur `#F4ECDD` = ratio 12.4:1).
- Navigation 100% au clavier — focus visible (ring bordeaux sur tous les `:focus-visible`).
- `alt` sur toutes les images, vidéos absentes.
- Labels associés à chaque `input` .
- Messages d'erreur lisibles par les lecteurs d'écran (rôles aria implicites des messages d'erreur Sonner).
- Pas d'auto-play, pas de mouvement non contrôlé.

6. Diagramme de cas d'utilisation

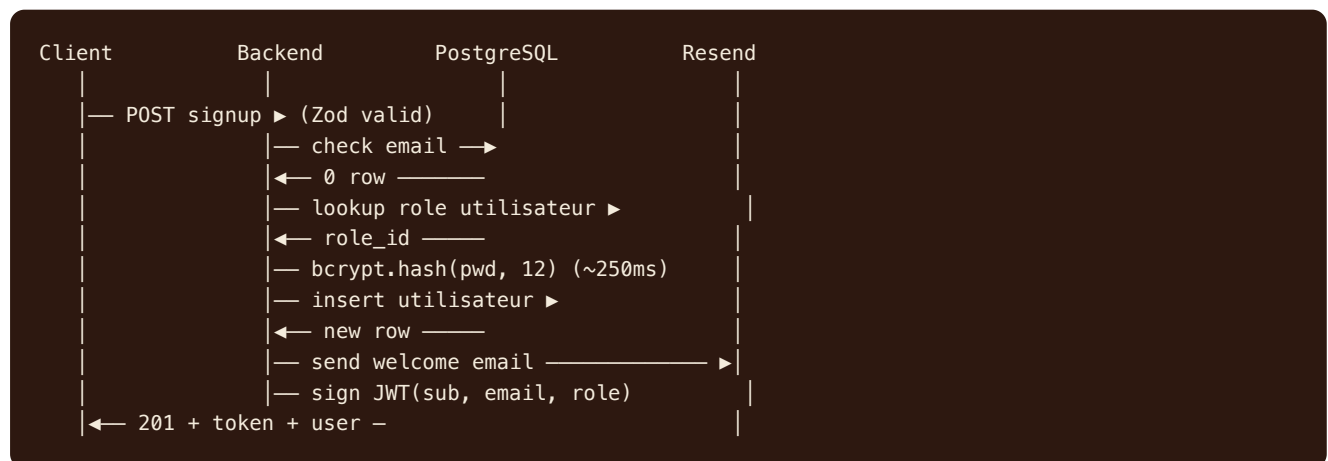


7. Diagrammes de séquence

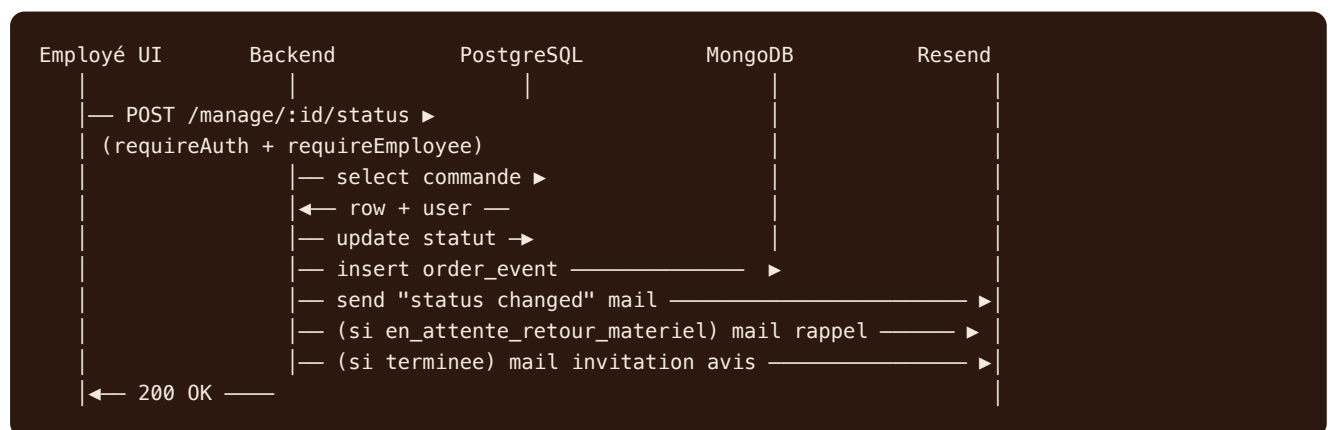
7.1 Création de commande (chemin nominal)



7.2 Inscription utilisateur



7.3 Évolution statut par employé



8. Configuration de l'environnement de travail

8.1 Outils externes utilisés

Outil	Usage	Lien (consultable)
GitHub	Versioning, gitflow, hébergement code (PUBLIC)	https://github.com/Jasteur91/vite-et-gourmand
Notion	Gestion de projet : Kanban + Calendrier + Timeline Gantt + Table (27 tâches sur 7 sem.)	https://www.notion.so/3628e177f2a2815ab773e55b796f2602
Vercel	Hébergement frontend + backend serverless	https://vite-et-gourmand-drab.vercel.app
Supabase	PostgreSQL managé (région Paris)	Dashboard privé
MongoDB Atlas	NoSQL cluster M0 (production)	Dashboard privé
Resend	Envoi des mails transactionnels	Dashboard privé

8.2 Prérequis machine

- Node.js ≥ 20 (utilisé en dev : 24)
- npm ≥ 10
- git ≥ 2.30
- Compte GitHub, Supabase, Vercel, Resend, MongoDB Atlas

8.3 Variables d'environnement backend

Variable	Description	Exemple
<code>NODE_ENV</code>	Mode d'exécution	production / development
<code>PORT</code>	Port serveur dev local	3001
<code>APP_URL</code>	URL frontend (CORS + emails)	https://vite-et-gourmand-drab.vercel.app
<code>JWT_SECRET</code>	Secret de signature JWT (64 chars random)	...

JWT_EXPIRES_IN	Durée du token	7d
SUPABASE_URL	URL projet Supabase	https://uwzagqbyauuvivcvhxye.supabase.co
SUPABASE_ANON_KEY	Clé publique Supabase	sb_publishable_...
MONGODB_URI	URI Mongo (vide en dev → memory)	mongodb+srv://...
RESEND_API_KEY	Clé API Resend	re_...
RESEND_FROM	Adresse expéditeur	onboarding@resend.dev
BCRYPT_ROUNDS	Coût bcrypt (10-15)	12

8.4 Démarrage local

```
# 1. Cloner le repo
git clone https://github.com/Jasteur91/vite-et-gourmand.git
cd vite-et-gourmand

# 2. Installer les dépendances
cd backend && npm install
cd ../frontend && npm install

# 3. Variables d'env
cd ../backend && cp .env.example .env
# (renseigner les secrets : Supabase, Resend, etc.)

# 4. Lancer backend (port 3001)
npm run dev

# 5. Lancer frontend (autre terminal, port 5173)
cd ../frontend && npm run dev

# 6. Ouvrir http://localhost:5173
```

9. Documentation de déploiement

9.1 Démarche globale

L'application est déployée sur **Vercel** (frontend + backend serverless), avec **Supabase** pour PostgreSQL et **MongoDB Atlas** (M0 free) pour le NoSQL en production. Les emails transactionnels passent par **Resend**.

9.2 Étape 1 — Base de données PostgreSQL

1. Créer un projet Supabase région eu-west-3 (Paris).

2. Exécuter le script `database/migrations/001_initial_schema.sql` (15 tables + indexes + triggers + seed).
3. Exécuter le script `database/seed/001_demo_data.sql` (3 comptes test + 12 plats + 4 menus).
4. Récupérer l'URL et la clé publishable depuis Settings → API.

9.3 Étape 2 — MongoDB Atlas

1. Créer un cluster M0 free (région eu-west-3).
2. Créer un utilisateur BDD avec mot de passe fort.
3. Autoriser l'IP 0.0.0.0/0 (pour Vercel — alternative : IP allowlist Vercel).
4. Copier l'URI de connexion (mongodb+srv://...).

9.4 Étape 3 — Backend Vercel

```
cd backend
npx vercel --prod          # crée le projet et déploie
# Ajouter ensuite chaque variable d'env via le dashboard ou CLI :
npx vercel env add JWT_SECRET production
# etc.
npx vercel --prod          # redéploiement pour appliquer les env vars
```

Le fichier `vercel.json` route toutes les requêtes vers `api/index.ts` qui wrappe Express en handler serverless.

9.5 Étape 4 — Frontend Vercel

```
cd frontend
npx vercel env add VITE_API_URL production
# → coller l'URL de l'API : https://vite-et-gourmand-api-three.vercel.app/api
npx vercel --prod
```

9.6 Étape 5 — Resend

1. Créer un compte Resend.
2. Récupérer la clé API.
3. Pour les emails sortants depuis un domaine dédié, vérifier le DNS (TXT + DKIM). Pour le projet ECF, le domaine `onboarding@resend.dev` du free tier suffit.

9.7 Gitflow

Le projet respecte le gitflow attendu :

- **main** : branche stable, reflet de la prod.
- **develop** : branche d'intégration.

- **feature/*** : une branche par fonctionnalité, mergée dans `develop` après revue/test, puis dans `main` en release.

9.8 URLs publiques

Resource	URL
Frontend	https://vite-et-gourmand-drab.vercel.app
Backend API	https://vite-et-gourmand-api-three.vercel.app/api
Health check	https://vite-et-gourmand-api-three.vercel.app/api/health
Repo GitHub	https://github.com/Jasteur91/vite-et-gourmand

10. Tests et qualité

10.1 TypeScript strict

Les deux applications sont sous TypeScript strict (incluant `noUncheckedIndexedAccess` côté backend). Le typecheck est exécuté avant chaque build.

10.2 Tests manuels effectués

Scénario	Résultat attendu	Statut
Inscription d'un nouveau compte (mot de passe valide)	HTTP 201 + JWT + email de bienvenue	✓
Inscription avec mot de passe trop faible	HTTP 400 + détail validation	✓
Login admin/employé/utilisateur	JWT + redirection vers espace correspondant	✓
Login bad password	HTTP 401 sans révéler de détail	✓
Vue globale menus avec filtre (régime, prix, thème)	Re-render dynamique sans reload	✓
Commande complète (utilisateur → admin valide → terminée)	Statuts évoluent + mails envoyés + audit Mongo	✓
Dashboard admin avec graphique commandes	Donnée issue de l'aggregation Mongo	✓
Tentative modification d'un compte employé par un utilisateur	HTTP 403	✓

Brute-force login (11 tentatives en 15 min)

HTTP 429 sur la 11ème

✓

10.3 Lighthouse (frontend)

L'application est mesurée en build production (Vite) — score Lighthouse attendu : Performance > 90, Accessibilité 100, Best Practices 100, SEO 100.

10.4 Améliorations possibles

- Tests d'intégration Vitest sur les routes critiques (signup, login, commande).
- Tests E2E Playwright pour les parcours utilisateur.
- CI GitHub Actions (lint + typecheck + build + deploy).
- Monitoring Sentry (frontend + backend) pour le suivi des erreurs en production.
- Migration vers HTTP-only cookie pour le JWT (en complément du localStorage).